

For Loops In Java

Generated Aug 13, 2026 8:10 AM

What We'll Learn

By the end of this lesson, students can:

- Explain how a for loop repeats statements.
- Identify the initialization, condition, and update parts of a for loop.
- Create loops that count upward and downward.
- Use a for loop to perform repeated calculations.
- Trace the values of a loop-control variable.

Before We Start

In the previous lessons, we used loops to repeat statements while a condition remained true.

Consider the following while loop:

```
int number = 1;

while (number <= 5) {
    System.out.println(number);
    number++;
}
```

The initialization, condition, and update are written in different parts of the program.

Is there a way to place these three parts together?

Let's Understand

A for loop repeats statements a specific number of times.

It is commonly used when the program knows how many repetitions are needed.

Basic Syntax

```
for (initialization; condition; update) {  
    // Statements to repeat  
}
```

A for loop contains three main parts:

```
for (int number = 1; number <= 5; number++) {  
    System.out.println(number);  
}
```

Part	Code	Purpose
Initialization	<code>int number = 1</code>	Creates the loop variable and gives it a starting value
Condition	<code>number <= 5</code>	Determines whether the loop should continue
Update	<code>number++</code>	Changes the loop variable after every repetition

How A For Loop Works

1. Java performs the initialization.
2. Java checks the condition.
3. If the condition is `true`, the loop body executes.
4. Java performs the update.
5. The condition is checked again.
6. The process continues until the condition becomes `false`.

The initialization is performed only once. The condition and update are performed repeatedly.

Counting Upward

Use the increment operator `++` to increase the loop variable.

```
for (int number = 1; number <= 5; number++) {  
    System.out.println(number);  
}
```

Output:

```
1
2
3
4
5
```

Counting Downward

Use the decrement operator -- to decrease the loop variable.

```
for (int number = 5; number >= 1; number--) {
    System.out.println(number);
}
```

Output:

```
5
4
3
2
1
```

Changing The Loop Variable By Another Amount

The loop variable does not always have to increase or decrease by one.

```
for (int number = 2; number <= 10; number += 2) {
    System.out.println(number);
}
```

Output:

```
2
4
6
8
10
```

The statement `number += 2` increases the value by two after every repetition.

For Loop And While Loop Comparison

The following loops produce the same output.

Using a while loop:

```
int number = 1;

while (number <= 5) {
    System.out.println(number);
    number++;
}
```

Using a for loop:

```
for (int number = 1; number <= 5; number++) {
    System.out.println(number);
}
```

A for loop is usually more convenient when the number of repetitions is known.

A while loop is often useful when the number of repetitions depends on a condition or user input.

Let's Practice

Practice 1: Trace The Loop

Study the following program:

```
public class LoopTracing {
    public static void main(String[] args) {
        for (int number = 3; number <= 15; number += 3) {
            System.out.println(number);
        }
    }
}
```

Answer the following questions:

1. What is the starting value of number?
2. What condition controls the loop?
3. How does the value of number change?
4. What numbers will the program display?
5. What is the value of number when the loop stops?

Practice 2: Complete The Loop

Complete the missing parts so the program displays the numbers from 1 to 10.

```
public class Completeloop {
    public static void main(String[] args) {
        for (int number = ____; number ____ 10; number____) {
            System.out.println(number);
        }
    }
}
```

Practice 3: Create A Countdown

Create a for loop that displays the following output:

```
10
9
8
7
6
5
4
3
2
1
Finished!
```

Practice 4: Display Multiples Of Five

Create a Java program that:

1. Uses a for loop.
2. Displays the multiples of 5 from 5 to 50.
3. Displays one number on each line.
4. Uses an appropriate initialization, condition, and update.

Practice 5: Find And Correct The Error

The following program is supposed to display the numbers from 1 to 5.

```
public class FixTheLoop {  
    public static void main(String[] args) {  
        for (int number = 1; number <= 5; number--) {  
            System.out.println(number);  
        }  
    }  
}
```

Answer the following:

1. What is wrong with the update statement?
2. What problem will the error cause?
3. Rewrite the loop correctly.

Let's Look at Examples

Example 1: Counting From One To Ten

This program displays the numbers from 1 to 10.

```
public class CountToTen {  
    public static void main(String[] args) {  
        for (int number = 1; number <= 10; number++) {  
            System.out.println(number);  
        }  
    }  
}
```

Common mistake

Students may use `number < 10`, which stops the loop before displaying 10.

Example 2: Displaying A Message Five Times

This program displays a message five times.

```
public class RepeatMessage {  
    public static void main(String[] args) {  
        for (int count = 1; count <= 5; count++) {  
            System.out.println(count + ". Keep learning Java!");  
        }  
    }  
}
```

Common mistake

Do not place a semicolon immediately after the for statement.

Example 3: Countdown

This program counts backward from 5 to 1.

```
public class Countdown {
    public static void main(String[] args) {
        for (int number = 5; number >= 1; number--) {
            System.out.println(number);
        }

        System.out.println("Start!");
    }
}
```

Example 4: Displaying Even Numbers

This program displays the even numbers from 2 to 20.

```
public class EvenNumbers {
    public static void main(String[] args) {
        for (int number = 2; number <= 20; number += 2) {
            System.out.println(number);
        }
    }
}
```

Example 5: Finding The Sum Of Numbers

This program finds the sum of the numbers from 1 to 5.

```
public class NumberSum {
    public static void main(String[] args) {
        int total = 0;

        for (int number = 1; number <= 5; number++) {
            total = total + number;
        }

        System.out.println("Total: " + total);
    }
}
```


Example 6: Creating A Multiplication Table

This program asks the user for a number and displays its multiplication table.

```
import java.util.Scanner;

public class MultiplicationTable {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int number = input.nextInt();

        for (int multiplier = 1; multiplier <= 10; multiplier++) {
            int product = number * multiplier;

            System.out.println(
                number + " x " + multiplier + " = " + product
            );
        }

        input.close();
    }
}
```

Resources / References

- [Java Development Kit](#)
- [Visual Studio Code](#)
- [Java for loop documentation](#)
- [Java Scanner class](#)